Candidate Table Builder

Architecture & Developer Guide

How the application turns a recruiter tracker and a set of resumes into a client-ready Excel table, and how to run, deploy and extend it.

Document date	19 October 2026
Applies to	Repository “Candidate-Table-Builder”, branch main
Audience	Developers who will maintain or extend the project
Sample data	All names, numbers and resumes in the screenshots are fictional

Contents

1. Overview 4
   1.1 What the output contains 4
   1.2 Design principles 4
2. Technology stack 5
3. Architecture 6
   3.1 Responsibilities 6
4. Repository structure 7
5. End-to-end flow 8
   5.1 Step by step 8
6. Frontend 10
   6.1 Screens and state 10
   6.2 Reading a resume in the browser 10
   6.3 Batching and pacing 10
   6.4 Progress display 11
   6.5 Review screen 12
7. Backend API 13
   7.1 Tracker parsing (tracker.py) 13
8. Gemini extraction 14
   8.1 What Gemini returns 14
   8.2 Reliability: retries, fallback and time budget 14
   8.3 Quota counter (usage.py) 15
9. Business rules 15
   9.1 Rules per column 15
   9.2 Years of experience 16
   9.3 Companies (Last 3) 16
10. Matching and validation 16
11. Verification tools 17
   11.1 Verify panel 17
   11.2 Verification sheet 19
12. Excel output format 19
13. Configuration 19
14. Running the project 20
   14.1 One-click start (Windows) 20
   14.2 Developer mode 20
15. Deployment 21
16. Limits and known constraints 21

17. How to extend the project 22
17.1 Add an output column that comes from the tracker 22
17.2 Add an output column that comes from the resume 22
17.3 Other common changes 23
18. Testing and troubleshooting 23
18.1 Regression check 23
18.2 Troubleshooting 24
19. Glossary 24

1. Overview

Candidate Table Builder is a web application for recruiters. A recruiter uploads two things:

- the **tracker** — an Excel file with one row per candidate (contact details, current and expected CTC, notice period, offers, comments);
- the candidates' **resumes** — 2 to 50 files, PDF or Word (.docx), including scanned PDFs.

The application reads each resume, pulls out the facts a client wants to see, pairs every resume with the right tracker row, and produces one formatted Excel table that can be sent to the client. Before downloading, the recruiter reviews the table and can check every value against its source.

1.1 What the output contains

Output column	Source	Example
Name	Resume	Arjun Mehta
No. of Years of Experience	Resume (tracker as fallback)	6+ Years
Education (Highest)	Resume	M.Tech - Data Science, National Institute of Technology Karnataka (2019)
Companies (Last 3)	Resume	1. Northwind Analytics – Senior Data Engineer | Mar 2024 – Present
Contact No	Resume, cross-checked with tracker	9811200341
Email id	Resume, cross-checked with tracker	arjun.mehta.dev@example.com
Current CTC (Fixed) + Any variable	Tracker	18 Lpa (Fixed) + 2 Lpa (Variable)
Expected CTC (Fixed)	Tracker	26 Lpa
Notice Period/Offer status	Tracker	60 Days (with an offer: 14 Days | Have an offer 20 Lpa; DOJ: 26th Oct'26)
Recruiter_Comments	Tracker	Open to relocate

1.2 Design principles

- **The AI only extracts facts.** Gemini returns raw facts (names, dates, degrees) as JSON. Every formatting rule and every calculation is plain Python code, so the output is consistent and testable.
- **Never mix up candidates.** Each resume is matched to a tracker row on phone, email and name. Anything uncertain is flagged instead of guessed.
- **Nothing is stored on the server.** Files are processed in memory for the length of one request.
- **Runs on free tiers.** Vercel (hosting) and Gemini (AI) free plans, with pacing and model fallback to stay inside their limits.

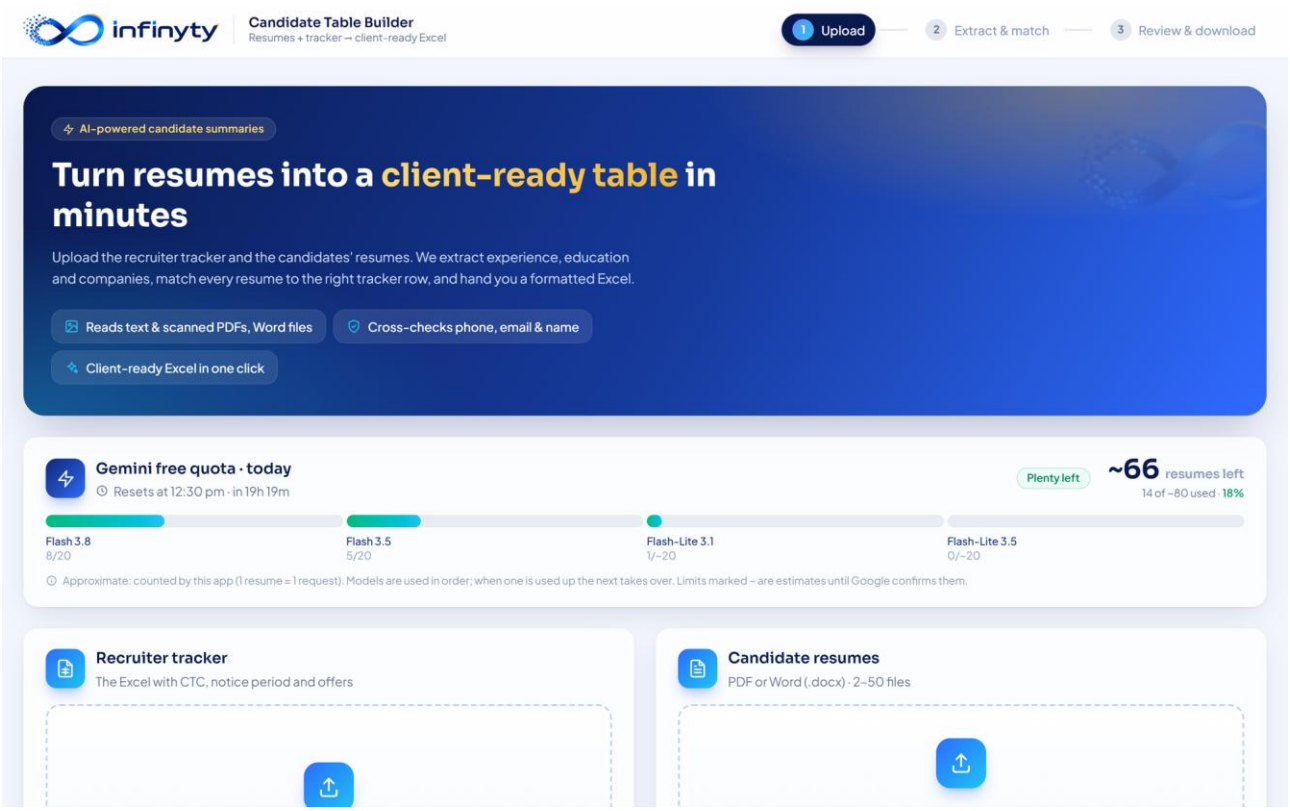


Figure 1. The home screen: banner, Gemini quota bar, and the two upload areas.

2. Technology stack

Layer	Technology	Why it is used
Frontend	React 18, TypeScript, Vite 6	Single-page app; fast builds; type safety across the UI.
Styling	Tailwind CSS 4 (theme in `index.css`)	Brand colours and animations defined once as theme tokens.
PDF reading	pdf.js (`pdfjs-dist` 4)	Extracts text in the browser and draws pages in the Verify panel.
Word reading	mammoth	Extracts text / HTML from .docx files in the browser.
Backend	Python, FastAPI	Small HTTP API; Python has the best Excel and text-matching libraries.
Excel	openpyxl	Reads the tracker and writes the formatted output workbook.
Matching	rapidfuzz	Fast fuzzy comparison of names, companies and institutes.
AI	Google Gemini API (flash models) over HTTPS with `httpx`	Reads text and scanned PDFs; returns structured JSON.
Hosting	Vercel (services: `web` + `api`)	Free plan hosts the static site and the Python API together.
Usage counter	Local JSON file, or Upstash Redis when configured	Tracks the daily Gemini quota.

3. Architecture

The system has two parts that talk over HTTP: a React application that runs entirely in the user's browser, and a stateless FastAPI backend. The backend calls the Gemini API and keeps a small counter of the day's usage.

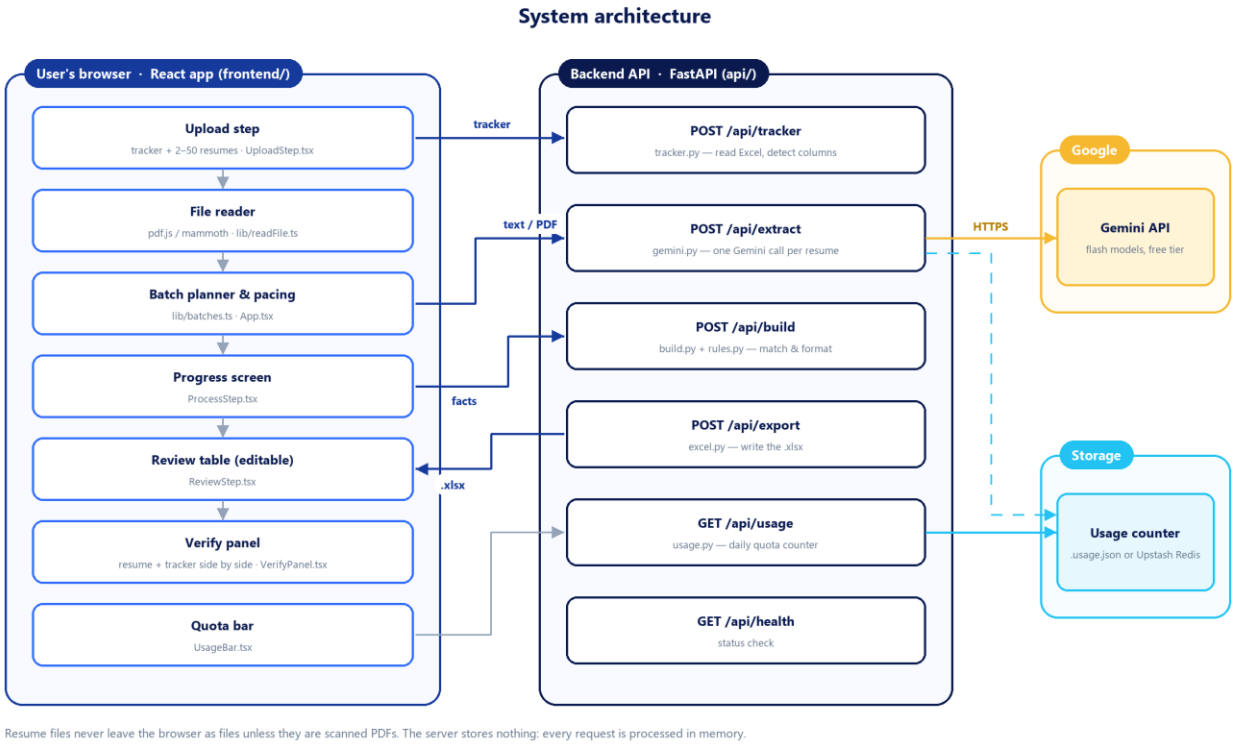


Figure 2. System architecture: browser components, backend endpoints and external services.

3.1 Responsibilities

Part	Responsible for	Not responsible for
Browser (React)	Reading files, deciding text vs. scanned, batching and pacing, showing progress, the editable review table, the Verify panel, triggering the download.	Any formatting rule or matching decision.
Backend (FastAPI)	Parsing the tracker, calling Gemini, matching resumes to tracker rows, applying every formatting rule, writing the Excel file, counting quota.	Storing files or results. Each request is independent.
Gemini	Extracting raw facts from one resume and returning JSON that follows a fixed schema.	Formatting, calculating experience, choosing which companies to show.

> **Why the split matters**
>
> Because rules live only in `api/_core/rules.py` and `api/_core/build.py`, a rule change is a Python change with no UI work, and the same resume always produces the same row for the same extracted facts.

4. Repository structure

```
Candidate-Table-Builder/
├── api/                          Backend (FastAPI)
│   ├── index.py                  HTTP routes and request models
│   ├── requirements.txt          Python dependencies
│   └── _core/
│       ├── tracker.py            Read tracker Excel, detect columns by header text
│       ├── gemini.py             Prompt, JSON schema, Gemini call, retries, model fallback
│       ├── rules.py              Every formatting rule (experience, education, companies, CTC…)
│       ├── build.py              Match resumes to tracker rows and build the table
│       ├── excel.py              Write the output workbook (client sheet + Verification sheet)
│       └── usage.py              Daily Gemini quota counter
├── frontend/                     Frontend (React + Vite)
│   ├── index.html
│   ├── public/                   logo.webp, mark.png
│   └── src/
│       ├── App.tsx               Step control and the run orchestration (start → review)
│       ├── types.ts              Shared TypeScript types
│       ├── index.css             Tailwind theme, animations, viewer styles
│       ├── lib/
│       │   ├── api.ts            Calls to the backend
│       │   ├── readFile.ts       Read PDF / DOCX, decide text vs. scanned
│       │   ├── batches.ts        Batch sizes and pacing constants
│       │   ├── evidence.ts       Where each output value came from (Verify panel)
│       │   └── audit.ts          Rows for the Verification sheet
│       └── components/
│           ├── UploadStep.tsx    Tracker + resume upload, column mapping
│           ├── UsageBar.tsx      Gemini quota bar
│           ├── ProcessStep.tsx   Progress ring and resume queue
│           ├── ReviewStep.tsx    Editable review table, download
│           ├── VerifyPanel.tsx   Output vs. source, verified ticks
│           ├── ResumeViewer.tsx  PDF / DOCX viewer with highlights
│           ├── TrackerViewer.tsx Tracker sheet grid with highlighted cells
│           └── ui.tsx, icons.tsx Shared building blocks
├── start.bat                     One-click local start on Windows
├── vercel.json                   Vercel services and rewrites
├── .env.example                  Template for the API key
└── README.md
```

Files that never go to git

`.env` (the API key), `.venv/`, `node_modules/`, `frontend/dist/`, every `*.xlsx` and `.usage.json` are listed in `.gitignore`.

5. End-to-end flow

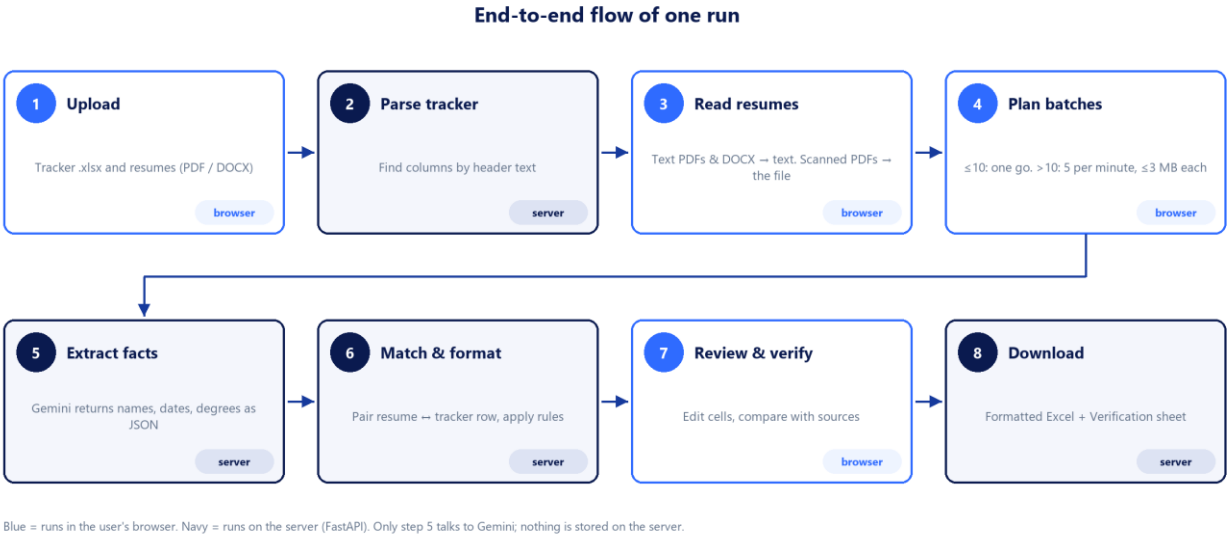


Figure 3. The eight steps of one run and where each one executes.

5.1 Step by step

#	Step	What happens	Code
1	Upload	The user drops the tracker and the resumes (files or a whole folder). Non PDF/DOCX files are skipped with a notice.	`UploadStep.tsx`
2	Parse tracker	The tracker is posted to `/api/tracker`. The server finds the header row and maps each needed field to a column by header text. The user can correct the mapping.	`tracker.py`
3	Read resumes	Each file is opened in the browser. Text is extracted; a PDF with too little text is treated as scanned and sent as a file instead.	`lib/readFile.ts`
4	Plan batches	Resumes are grouped into requests: all at once for 10 or fewer, otherwise 5 per minute; never more than about 3 MB per request.	`lib/batches.ts`, `App.tsx`
5	Extract facts	`/api/extract` calls Gemini once per resume (up to 5 in parallel) and returns the facts as JSON.	`gemini.py`
6	Match & format	`/api/build` pairs resumes with tracker rows, applies all formatting rules, and returns rows with a status and notes.	`build.py`, `rules.py`
7	Review & verify	The user edits cells, marks rows red, and compares values with the resume and the tracker.	`ReviewStep.tsx`, `VerifyPanel.tsx`
8	Download	`/api/export` writes the workbook. The browser names the file with the current date and time.	`excel.py`

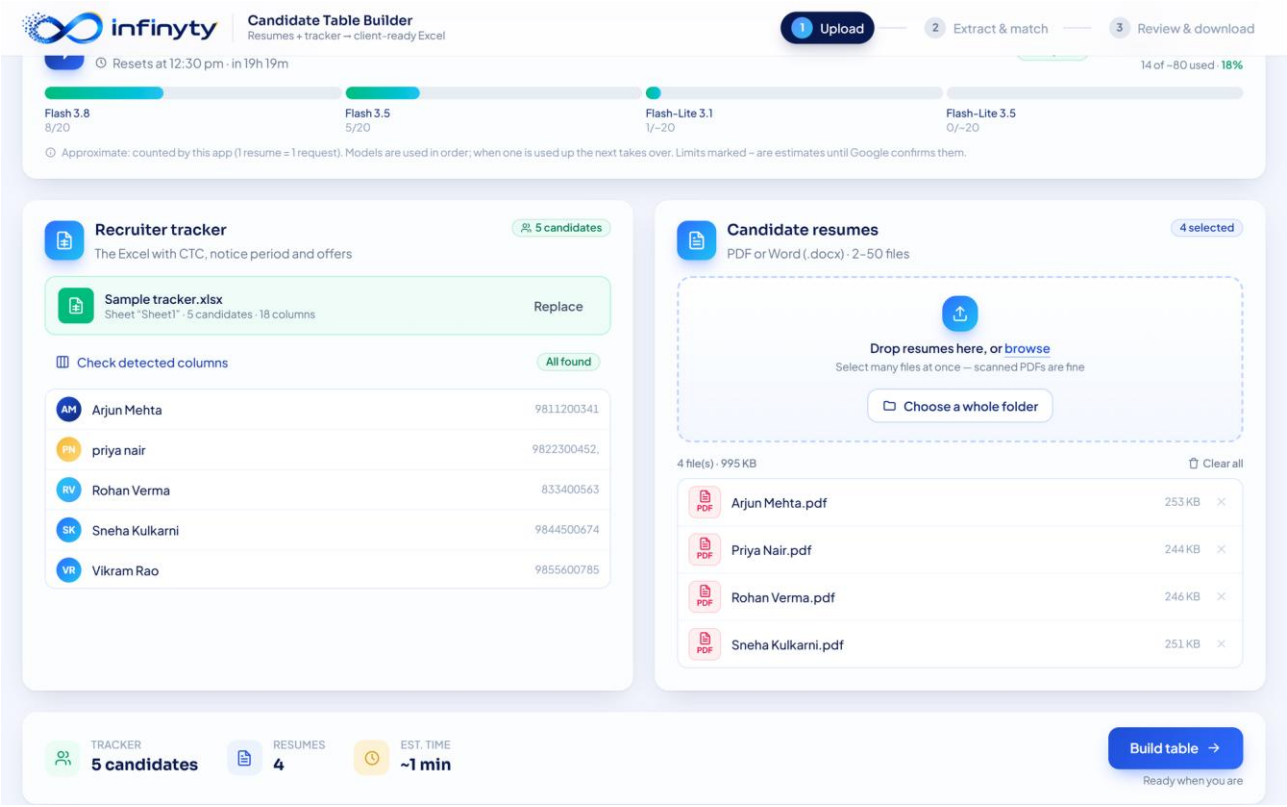


Figure 4. Upload screen with a tracker and four resumes loaded. The summary bar shows counts and the estimated time.

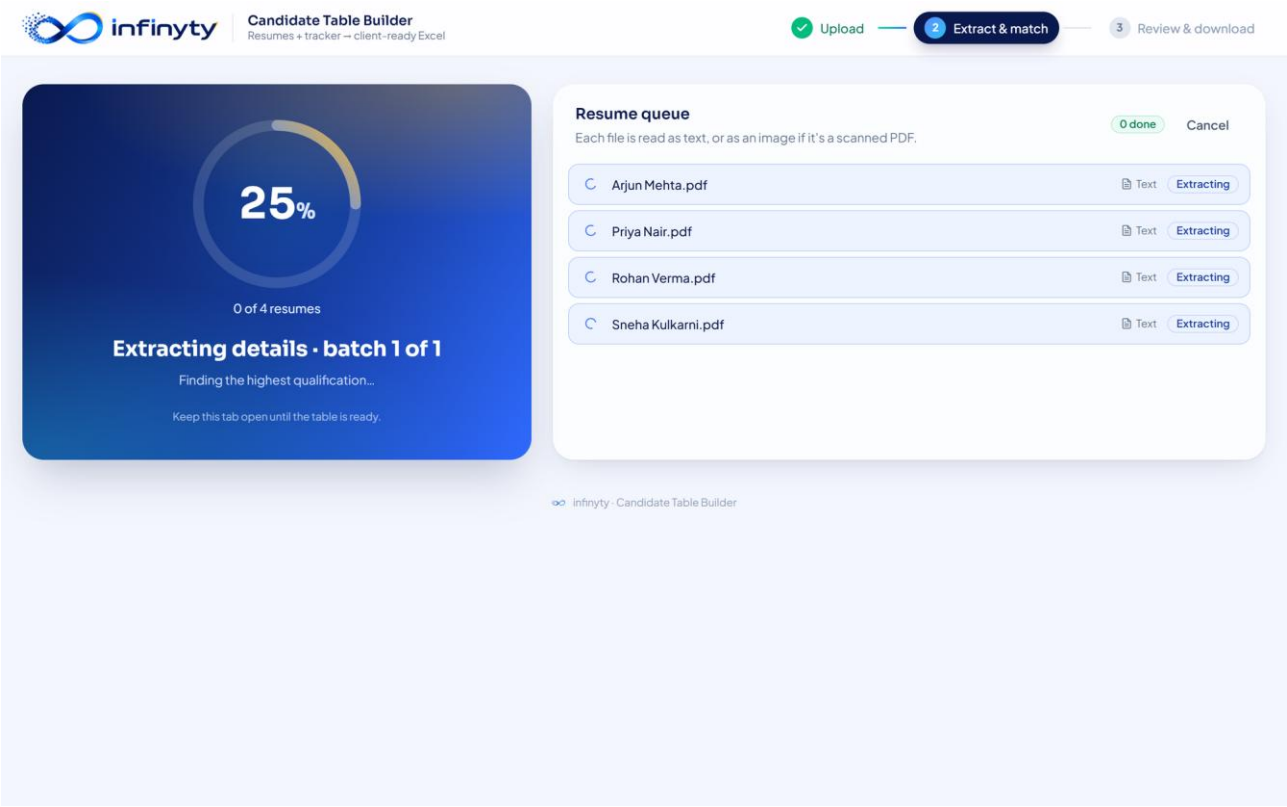


Figure 5. Processing screen: progress ring on the left, live status of each resume on the right.

6. Frontend

6.1 Screens and state

`App.tsx` owns the application state and switches between three steps: **upload**, **process** and **review**. There is no router and no global state library; state is passed to the step components as props.

State in App.tsx	Meaning
`tracker`, `mapping`	Parsed tracker (headers, rows, row numbers) and the field → column mapping.
`files`	The resume files the user selected.
`jobs`	One entry per resume during a run: file, read mode, extracted text, Gemini result, status.
`run`	Phase of the run (reading, extracting, waiting, building, done, error), batch number, countdown target.
`table`	The result: columns and rows. Each row carries values, status, notes, the tracker row index, `highlight` and `verified`.
`elapsedMs`	How long the run took, shown on the review screen.
`usage`	Gemini quota summary for the quota bar.

6.2 Reading a resume in the browser

Reading happens in the browser so that only a few kilobytes of text travel to the server, and the original file stays on the user's machine whenever possible.

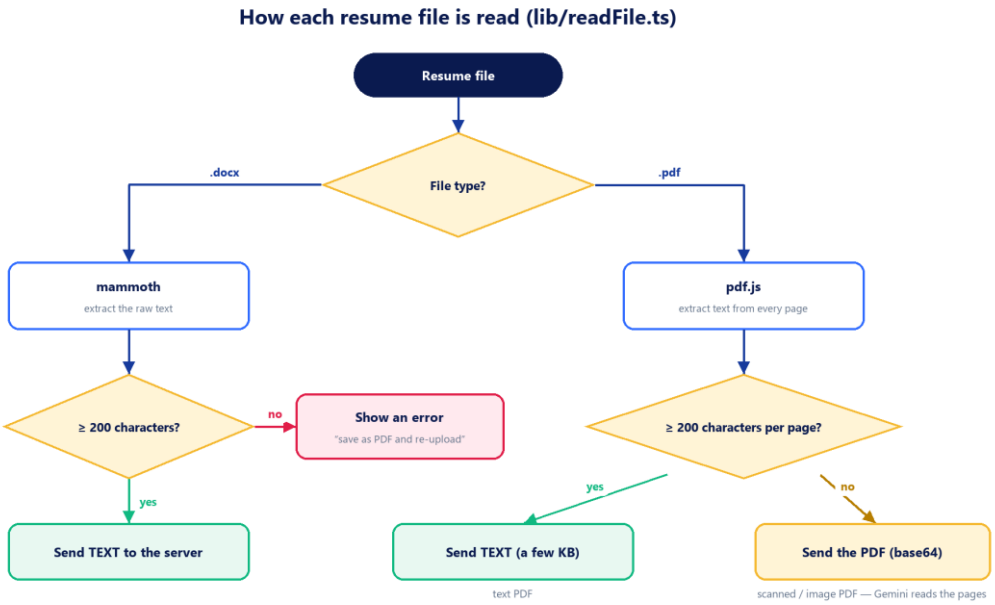


Figure 6. Decision made for every file in lib/readFile.ts.

6.3 Batching and pacing

Gemini's free tier limits requests per minute, and Vercel limits a request body to 4.5 MB. The browser therefore controls the timing. Constants are at the top of `lib/batches.ts`.

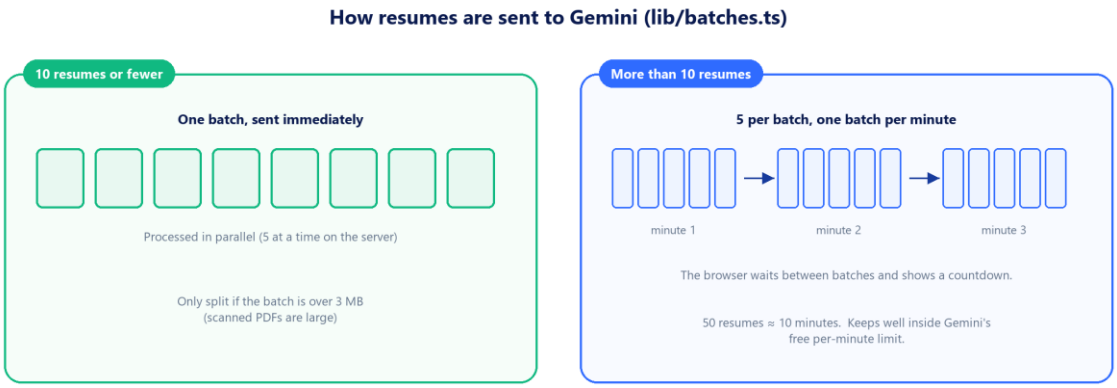


Figure 7. Small runs go at once; larger runs are paced at five resumes per minute.

Constant	Value	Meaning
`SMALL_RUN_LIMIT`	10	Up to this many resumes are sent without waiting.
`PACED_BATCH_SIZE`	5	Batch size for larger runs.
`PACED_INTERVAL_MS`	60 000	Time from the start of one batch to the start of the next.
`MAX_BATCH_BYTES`	3 MB	Upper size of one request (text length or base64 length).
`MIN_RESUMES` / `MAX_RESUMES`	2 / 50	Allowed number of resumes per run (`UploadStep.tsx`).

6.4 Progress display

All resumes of a batch return together, so the true count would jump from 0% to 100%. `ProcessStep.tsx` shows an estimate instead: reading 5%, extraction 85%, matching 10%. While a batch is with Gemini, its share fills gradually and is capped below 100% until the results arrive. The value never moves backwards, and the animation uses timers so it keeps moving in a background tab.

6.5 Review screen

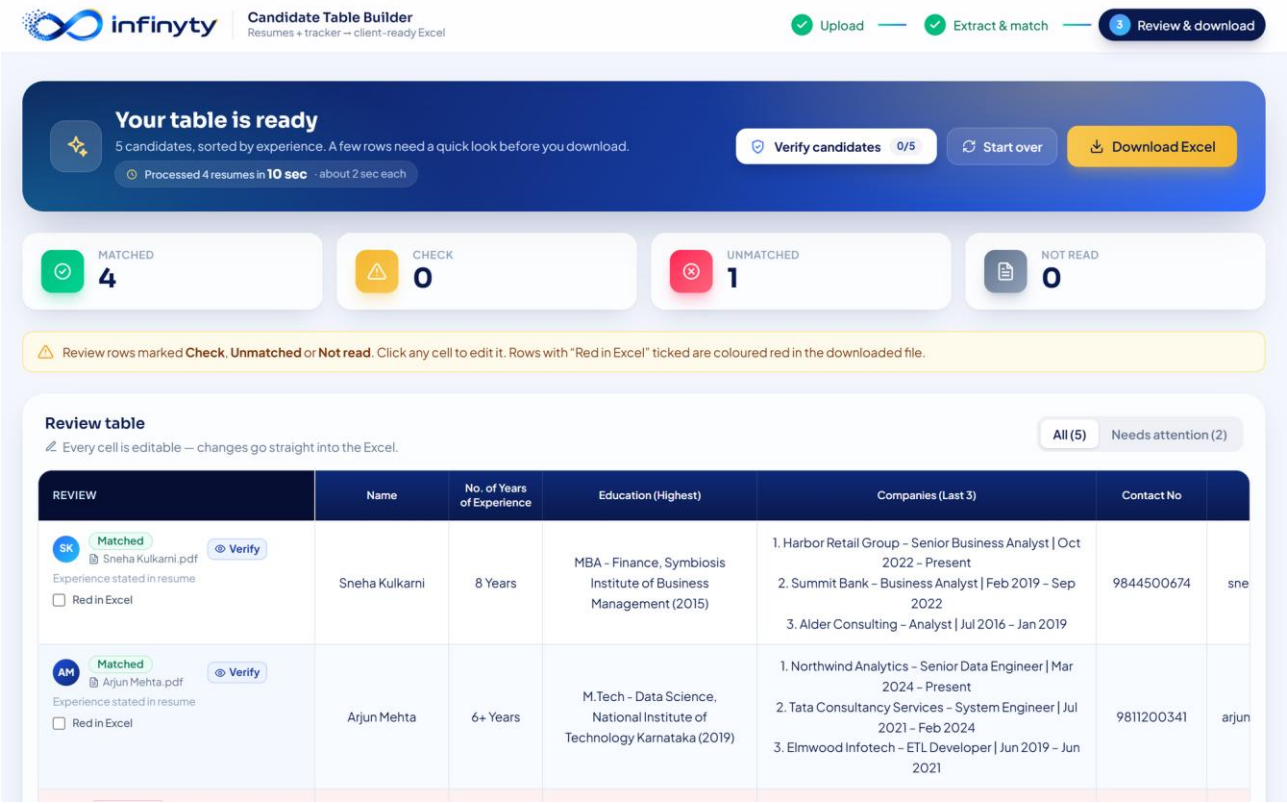


Figure 8. Review screen: processing time, status counts, and the editable table.



Figure 9. The review table. The red row is a tracker candidate with no resume; it will be red in the Excel as well.

- Every cell is editable (`contentEditable`); edits are stored in `table.rows[].values` and go straight into the export.
- "Red in Excel" controls the `highlight` flag. It is pre-ticked for Unmatched and Not read rows.

- “Needs attention” filters to rows that are not cleanly matched or that carry notes.

7. Backend API

The backend is one FastAPI application defined in `api/index.py`. All routes live under `/api`. Interactive documentation is available at `/api/docs` when the server is running.

Method & path	Input	Output	Module
GET `/api/health`	—	`ok`, current model, whether the API key is configured.	`index.py`
GET `/api/usage`	—	Per-model usage and limits, total remaining, reset time.	`usage.py`
POST `/api/tracker`	Multipart file (.xlsx)	`headers`, `rows`, `row_numbers`, detected `mapping`, field labels, required fields.	`tracker.py`
POST `/api/extract`	JSON: up to 20 resumes, each `mode` = `text` (with `text`) or `file` (with `file_b64`).	For each resume: `data` (facts) or `error`.	`gemini.py`
POST `/api/build`	JSON: resumes with their facts and text, tracker rows, mapping.	`columns` and `rows`; each row has `values`, `status`, `notes`, `match`, `tracker_row`, `experience_source`.	`build.py`, `rules.py`
POST `/api/export`	JSON: columns, rows (`values`, `highlight`, `audit`), audit column names.	The .xlsx file.	`excel.py`

7.1 Tracker parsing (tracker.py)

Tracker column order changes between roles, so columns are never addressed by letter. The header row is the first of the top ten rows that contains a candidate-name header. Each field is matched by keywords in the header text:

Field key	Header must contain	Required
`name`	“candidate name” (or exactly “name”)	Yes
`phone`	“contact”, “phone” or “mobile”	Yes
`email`	“email”, “e-mail” or “mail id”	Yes
`current_ctc`	“current ctc”	Yes
`variable`	“variable”	Yes
`expected_ctc`	“expected ctc”	Yes
`notice`	“notice”	Yes
`offer`	“offer”	Yes
`total_exp`	“total” together with “exp”, “yrs” or “year”	No
`comments`	“comment” or “remark”	No

Only the first sheet of the workbook is read. Rows without a candidate name are dropped. The Excel row number of each kept row is returned in `row_numbers` so the Verify panel can show references such as “E5”.

8. Gemini extraction

`api/_core/gemini.py` sends one request per resume to the Gemini REST API with temperature 0 and a response schema, so the answer is always JSON in a known shape. For a text resume the text is sent; for a scanned PDF the PDF itself is sent inline (base64) and Gemini reads the pages.

8.1 What Gemini returns

Field	Content
`name`	Full name as written.
`phones`, `emails`	Every phone number and email address of the candidate.
`stated_total_experience`	The exact phrase stating total experience (for example “6+ years of experience”), or null.
`education[]`	`level` (phd, masters, bachelors, diploma, school, other), `degree`, `branch`, `institute`, `year`. Missing parts are null.
`experience[]`	One entry per role: `company`, `title`, `start_month`, `start_year`, `end_month`, `end_year`, `is_current`, `is_internship`, `is_client_project`.

> **Client projects**
> IT-services resumes often list client assignments under the employer (for example Tata Consultancy Services → Orion Media). Gemini marks those entries with `is_client_project = true` and the code leaves them out of “Companies (Last 3)”. Freelance work and overlapping jobs are kept. If every entry is flagged, the flags are ignored.

8.2 Reliability: retries, fallback and time budget

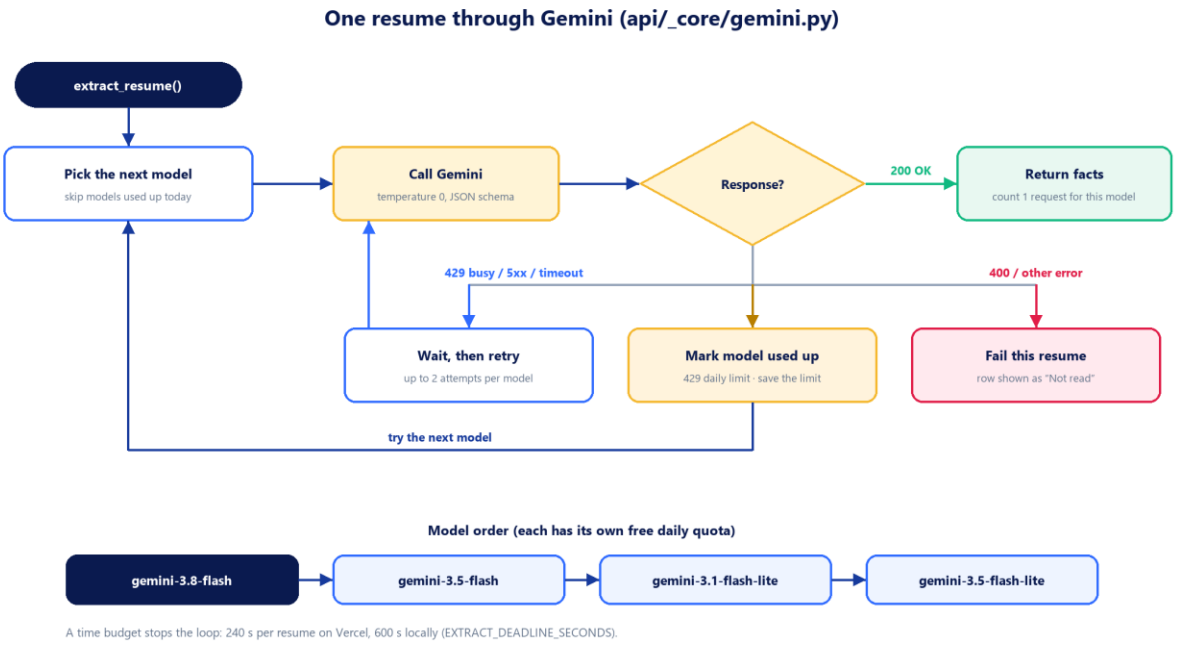


Figure 10. What happens to one resume inside extract_resume().

- **Retry:** busy (429 per-minute), 5xx and network errors are retried, up to 2 attempts per model, honouring the delay Gemini suggests.
- **Fallback:** when a model's free **daily** quota is used up, the next model in the chain is tried. Models already known to be used up today are skipped immediately.
- **Time budget:** each resume has a deadline (240 s on Vercel, 600 s locally) so a request can never exceed the hosting limit.
- **Failure:** if nothing works, the resume gets an `error`, and its row is shown as “Not read” with tracker data filled in.

8.3 Quota counter (usage.py)

Google has no API for remaining free quota, so the application counts its own successful calls per model for the current quota day (the day changes at midnight US Pacific time). When Google reports that a model's daily limit is reached, the model is marked used up and the limit Google quotes is saved, so the numbers correct themselves. The counter is stored in `.usage.json` locally, in `/tmp` on Vercel, or in Upstash Redis when `UPSTASH_REDIS_REST_URL` and `UPSTASH_REDIS_REST_TOKEN` are set.

9. Business rules

All rules are in `api/_core/rules.py` (formatting) and `api/_core/build.py` (choosing sources). This chapter is the reference for what the output must look like.

9.1 Rules per column

Column	Rule
Name	Name from the resume. Names written fully in upper or lower case are converted to title case. Falls back to the tracker name.
No. of Years of Experience	See 9.2. Shown as `4 Years` or `4+ Years`.
Education (Highest)	Highest level wins: PhD > Master's > Bachelor's > diploma. Format `Degree - Branch, Institute (Year)`, using only parts the resume states. Long degree names are shortened (Bachelor of Technology → B.Tech). City and state are removed from the institute.
Companies (Last 3)	See 9.3. One line per company: `1. Company – Title | Mon YYYY – Present`.
Contact No	10 digits, without +91 or a leading 0. A valid number from the resume wins; a difference from the tracker is noted.
Email id	Lower case, spaces removed. Resume first, tracker otherwise; a difference is noted.
Current CTC (Fixed) + Any variable	Only the fixed amount, e.g. `12.3 Lpa`, when variable is empty, NA, No, Nil or 0; otherwise `10 Lpa (Fixed) + 2 Lpa (Variable)`. A bare number gets “Lpa”.
Expected CTC (Fixed)	Copied from the tracker.
Notice Period/Offer status	The notice period as written, e.g. `60 Days`. Only when there is an offer: `<notice period> | <offer & DOJ>`. Any “LWD …” inside the offer cell is removed; an empty, NA or “No offer” cell counts as no offer. “offer of X” becomes “offer X”.
Recruiter_Comments	Copied from the tracker's comments column.

9.2 Years of experience

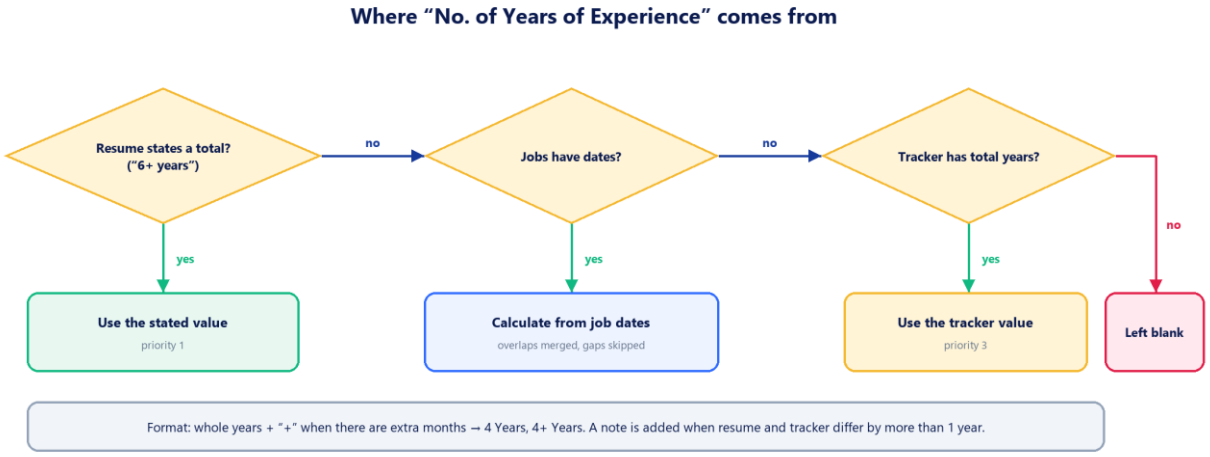


Figure 11. Priority of sources for the experience column.

When calculating from dates, every role becomes a month interval; overlapping intervals are merged and gaps between jobs are not counted. A current role runs to the current month. If the resume value and the tracker value differ by more than one year, a note is added to the row.

9.3 Companies (Last 3)

1. Client-project entries are removed (see 8.1).
2. Roles are ordered most recent first. If any role has no start year, the resume's own order is kept.
3. Consecutive roles at the same employer are grouped. Company names are compared without suffixes such as Pvt, Ltd, Technologies.
4. The three most recent companies get one line each: the latest title, and the date range of the whole stay.
5. If the candidate has fewer than three companies, the spare lines are used to list individual roles, most recent company first.

Candidate history	Output
Five companies	The three most recent companies, one line each.
One company, three roles	Three lines, one per role, all with the same company name.
Two companies, the first with two roles	Two lines for the first company's roles, one line for the second company.

10. Matching and validation

Matching is the step that prevents one candidate's CTC from landing on another candidate's row. It is implemented in `match()` and `build_table()` in `api/_core/build.py`.

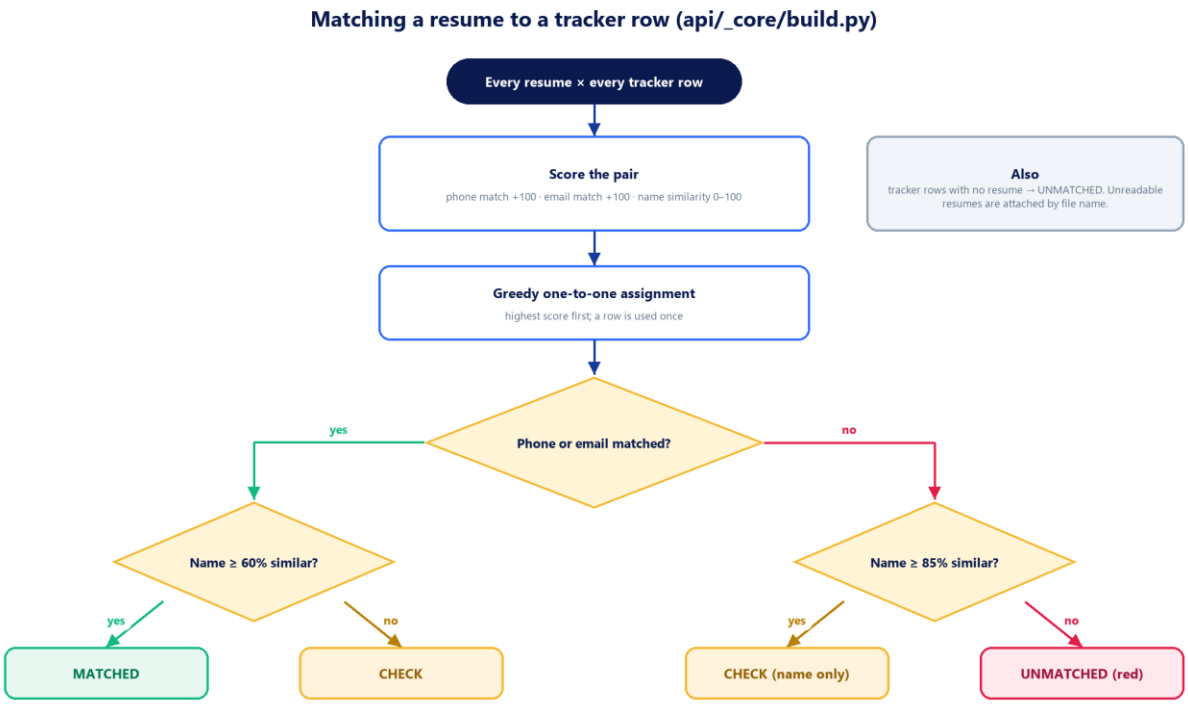


Figure 12. How a resume is paired with a tracker row and which status it receives.

Status	Meaning	In the Excel
Matched	Phone or email agrees and the names are similar.	Normal row
Check	Matched on name only, or phone/email agree but the names differ, or two resumes competed for one row.	Normal row (the user decides)
Unmatched	A resume with no tracker row, or a tracker row with no resume.	Red row
Not read	The resume could not be extracted. Tracker data is filled in if the file name matches a candidate.	Red row

- Phone comparison tolerates a tracker number with a missing or extra digit (for example 833400563 vs 8334005630).
- The file name is also compared with the tracker name, because resumes are usually saved under the candidate's name.
- For text resumes, a fact check confirms that the institute, graduation year and company names appear in the resume text; anything that does not is added to the row's notes.

11. Verification tools

11.1 Verify panel

The Verify panel puts the output next to its sources. The left side shows the resume or the tracker sheet; the right side shows one card per output column with its value, its source and a check result.

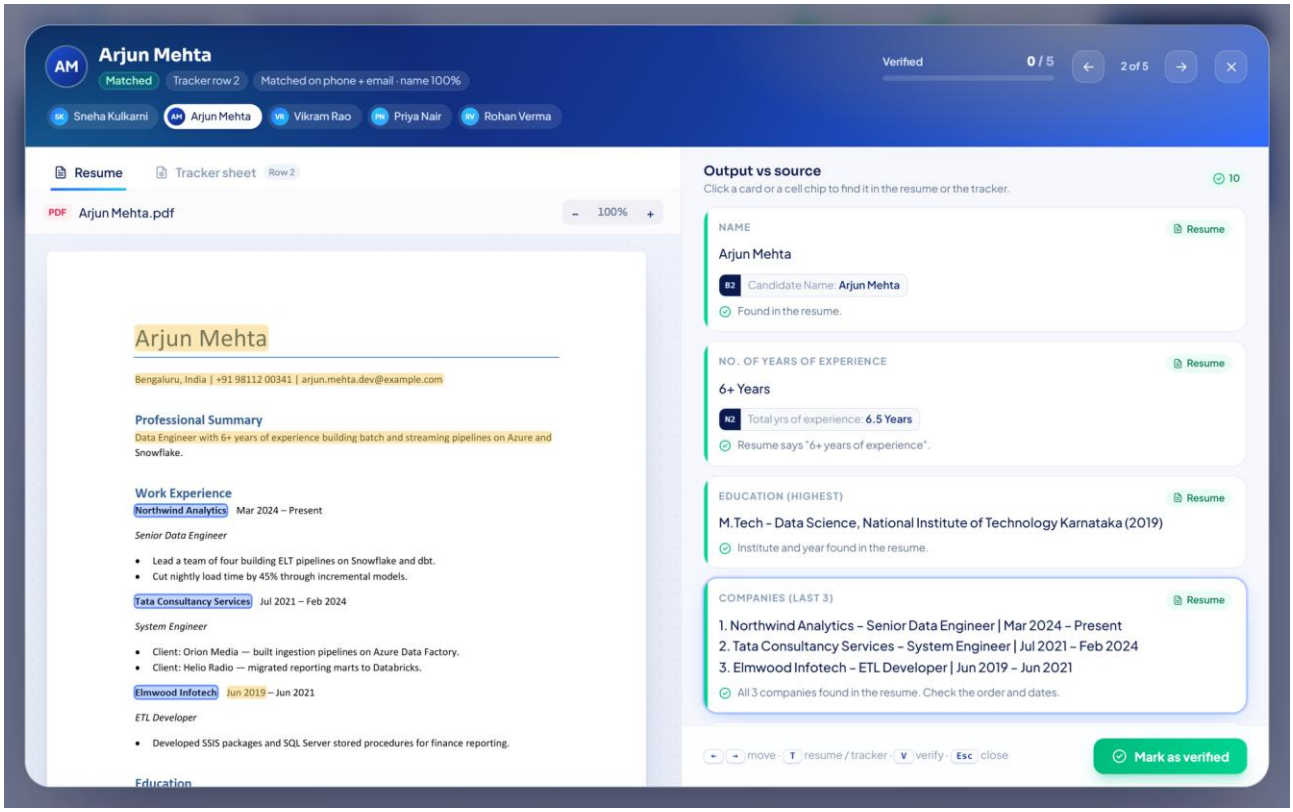


Figure 13. Verify panel, Resume tab. Extracted values are highlighted; clicking a card jumps to the value on the page.

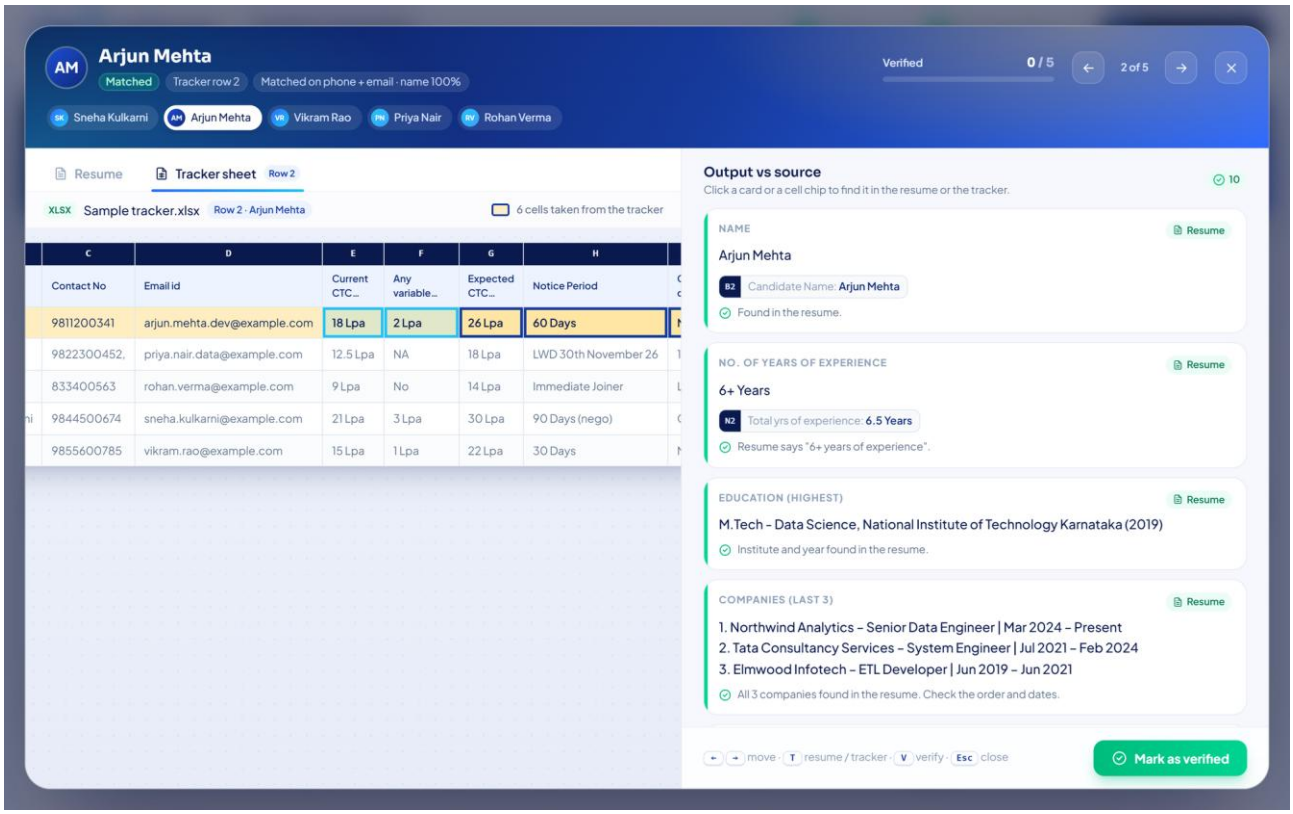


Figure 14. Verify panel, Tracker sheet tab. The candidate's row is gold; cells taken only from the tracker have a dark frame.

Piece	File	Role
Evidence	`lib/evidence.ts`	For each column: source (resume, tracker, calculated), status (ok, warn, info, missing), the words to highlight, and the tracker cells involved.

Piece	File	Role
Resume viewer	`ResumeViewer.tsx`	Draws PDF pages with pdf.js and a text layer, or a .docx as HTML, and highlights matching text.
Tracker viewer	`TrackerViewer.tsx`	Shows the whole sheet with column letters and row numbers; highlights the row and frames tracker-only cells.
Panel	`VerifyPanel.tsx`	Navigation, keyboard shortcuts (← → move, T switch tab, V verify, Esc close), verified ticks.

> **Scanned resumes**
> A scanned PDF has no text layer, so nothing can be highlighted and resume values are marked for a visual check. Tracker values are still compared.

11.2 Verification sheet

The downloaded workbook has two sheets. **Candidates** is the client table. **Verification** is for the recruiter and lists, for every row: resume file, how it was read, match result and evidence, tracker row, the raw tracker values, the experience source, the Gemini model used, whether the row was marked verified, and any notes. The rows are built in `lib/audit.ts` and written by `_audit_sheet()` in `excel.py`.

12. Excel output format

Aspect	Setting (api/_core/excel.py)
Sheet name	Candidates (first sheet), Verification (second sheet)
Header row	Bold white text on dark blue (`1F4E78`), centered, frozen
Cells	Centered horizontally and vertically, wrapped text, thin black borders, stored as text so phone numbers keep their digits
Column widths	`WIDTHS` dictionary, in Excel character units
Row heights	Estimated from the longest wrapped cell, because Excel does not auto-fit rows written by a library
Red rows	Light red fill (`FFC7CE`) with dark red text when `highlight` is true
File name	Set in the browser: `YYYY-MM-DD_HH-MM-SS.xlsx`
Row order	Most experienced first

13. Configuration

Configuration is read from environment variables. Locally they come from the `.env` file in the project root; on Vercel they are set in the project settings.

Variable	Default	Purpose
`GEMINI_API_KEY`	— (required)	Google AI Studio API key.
`GEMINI_MODEL`	`gemini-3.8-flash`	Main model.

Variable	Default	Purpose
`GEMINI_FALLBACK_MODELS`	`gemini-3.5-flash, gemini-3.1-flash-lite, gemini-3.5-flash-lite`	Comma-separated models tried when the main one is busy or out of daily quota.
`GEMINI_DAILY_LIMITS`	20 per model	Optional override of daily limits for the quota bar, e.g. `gemini-3.1-flash-lite:500`.
`EXTRACT_PARALLEL`	5	Resumes sent to Gemini at the same time within one request.
`EXTRACT_DEADLINE_SECONDS`	240 on Vercel, 600 locally	Time budget for one resume.
`UPSTASH_REDIS_REST_URL`, `UPSTASH_REDIS_REST_TOKEN`	not set	When both are set, the quota counter is stored in Upstash Redis and shared by all users.

Keep the key private

The API key belongs only in `.env` (ignored by git) or in Vercel's environment variables. Never commit it and never paste it into code.

14. Running the project

14.1 One-click start (Windows)

Double-click `start.bat`. It installs Python and Node.js if they are missing, creates or repairs the Python environment, builds the dashboard, asks for the Gemini key if there is no `.env`, and starts the app at `http://localhost:8000`. In this mode FastAPI also serves the built frontend from `frontend/dist`.

14.2 Developer mode

```
# Terminal 1 – API on port 8000, restarts on code changes
.venv\Scripts\python -m uvicorn api.index:app --reload --port 8000

# Terminal 2 – frontend on port 5173, hot reload; /api is proxied to port 8000
cd frontend
npm run dev
```

Remember to restart

A server started without `--reload` keeps running the old Python code. After a backend change, restart it; after a frontend change with `start.bat`, rebuild with `npm run build` and refresh the page.

15. Deployment

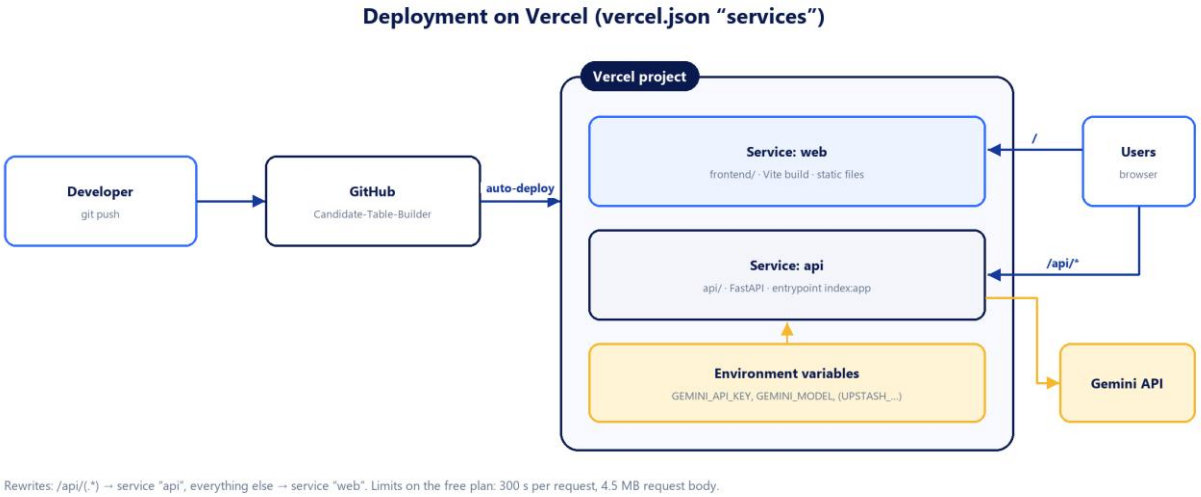


Figure 15. Deployment: GitHub pushes trigger a Vercel build of both services.

`vercel.json` declares two services in one project and routes requests between them:

```
{
  "services": {
    "web": { "root": "frontend/", "framework": "vite" },
    "api": { "root": "api/", "framework": "fastapi", "entrypoint": "index:app" }
  },
  "rewrites": [
    { "source": "/api/(.*)", "destination": { "service": "api" } },
    { "source": "/(.*)",     "destination": { "service": "web" } }
  ]
}
```

1. Push to the `main` branch on GitHub.
2. In Vercel, import the repository once as a single project (application preset "Services").
3. Add `GEMINI_API_KEY` and `GEMINI_MODEL` under Environment Variables.
4. Deploy. Later pushes redeploy automatically.
5. Check `https://<project>.vercel.app/api/health` — it should report `key_configured: true`.

16. Limits and known constraints

Area	Constraint	Effect / workaround
Gemini free tier	About 20 requests per model per day (measured on the flash models).	The fallback chain gives roughly 60–80 resumes a day. Do large runs early; lite models are less accurate.
Quota bar	An estimate. Calls made with the same key outside the app are not counted.	Limits correct themselves when Google reports a limit reached.
Quota counter	One counter, not separated per API key; on Vercel without Upstash it resets when the instance restarts.	Configure Upstash, or delete `.usage.json` after changing the key.

Area	Constraint	Effect / workaround
Vercel free plan	300 s per request, 4.5 MB request body. The Hobby plan is intended for non-commercial use.	Batches are capped at 3 MB; each resume has a 240 s budget.
Scanned PDFs	No text layer.	Gemini reads the pages; no highlights and no automatic text checks.
File types	PDF and .docx only. Old .doc files are not supported.	Save as PDF or .docx.
Tracker	Only the first sheet is read.	Put candidates on the first sheet.
Browser tab	The browser schedules the batches.	Keep the tab open during a run.
Access	No login or password.	Share the URL only with the team, or add authentication before wider use.
AI variation	Gemini may copy a title or company slightly differently between runs.	Rules normalise common cases; the review screen and Verify panel catch the rest.

17. How to extend the project

The tables below list the files to touch for the most likely changes. After any change to rules or extraction, run the regression check described in chapter 18.

17.1 Add an output column that comes from the tracker

Step	File	Change
1	`api/_core/tracker.py`	Add the field to `FIELDS` with a header matcher (and to `REQUIRED_FIELDS` if it must exist).
2	`api/_core/build.py`	Add the column name to `COLUMNS` and fill it in `tracker_part()`.
3	`api/_core/excel.py`	Add a width to `WIDTHS`.
4	`frontend/src/types.ts`	Add the key to `FieldKey`.
5	`frontend/src/components/ReviewStep.tsx`	Add a minimum width to `WIDTH`.
6	`frontend/src/lib/evidence.ts`	Add a `fromTracker(...)` line so the Verify panel shows the source cell.

17.2 Add an output column that comes from the resume

Step	File	Change
1	`api/_core/gemini.py`	Describe the new fact in `PROMPT` and add it to `SCHEMA` (and to `required`).
2	`api/_core/rules.py`	Write a function that formats the fact.
3	`api/_core/build.py`	Add the column to `COLUMNS` and set its value where resume values are filled.
4	`excel.py`, `ReviewStep.tsx`	Add widths, as in 17.1.

Step	File	Change
5	`frontend/src/lib/evidence.ts`	Add an evidence block with the terms to highlight.

17.3 Other common changes

Goal	Where
Change how a value is formatted	The matching function in `api/_core/rules.py`.
Change matching strictness	`NAME_STRONG` (85) and `NAME_WEAK` (60) in `api/_core/build.py`.
Use a different Gemini model	`GEMINI_MODEL` / `GEMINI_FALLBACK_MODELS` in `.env` or Vercel settings. No code change.
Use a different AI provider	Replace the body of `extract_resume()` in `gemini.py`. It must return the same facts dictionary (chapter 8.1).
Change batch size or pacing	Constants in `frontend/src/lib/batches.ts`.
Change the Excel look	Fills, fonts and `WIDTHS` at the top of `api/_core/excel.py`.
Change colours or animations	`@theme` block and keyframes in `frontend/src/index.css`.
Support another file type	`readResume()` and `isResumeFile()` in `frontend/src/lib/readFile.ts`, and the `accept` attribute in `UploadStep.tsx`.
Add a login	A FastAPI dependency on the routes in `api/index.py`, plus a sign-in screen in `App.tsx`.

18. Testing and troubleshooting

18.1 Regression check

The repository has no automated test suite yet. During development, changes were verified with a regression check that is worth keeping as a habit:

1. Keep one or two sample sets (tracker + resumes) outside the repository, because they contain personal data.
2. Extract each resume once and save the Gemini results to a JSON file. This avoids spending quota on every check.
3. After a change, run `build.build_table()` on the saved facts and compare every cell with the previous output.
4. Only the differences you intended should appear.

> **Recommended next step**
>
> Add `pytest` tests for `api/_core/rules.py`. The functions are pure (text in, text out), so cases such as `format_notice_offer("14 Days", "LWD: 20th Oct'26; Have an offer 20 Lpa")` are quick to write and protect the client format.

18.2 Troubleshooting

Symptom	Likely cause	Fix
“Row NaN” or empty tracker cells in the Verify panel	The backend is running old code.	Restart the server (close and re-run `start.bat`).
Rows show “Not read” with a daily-limit message	All Gemini models are out of free quota.	Wait for the reset shown on the quota bar, or use another key.
“Could not read the tracker”	The file is not a real .xlsx, or the first sheet is empty.	Re-save the tracker as .xlsx.
A required tracker column is not found	The header text changed.	Pick the column in “Check detected columns”, or extend the matcher in `tracker.py`.
Quota bar shows models as used up after changing the key	The counter belongs to the old key.	Delete `.usage.json` and restart.
`start.bat` says the port is in use	The app is already running.	Use the open browser tab, or close the other window.
A client name appears in Companies	Gemini did not flag a client project.	Edit the cell in the review table; adjust the prompt in `gemini.py` if it repeats.
`/api/health` shows `key_configured: false`	The key is missing.	Add `GEMINI_API_KEY` to `.env` or the Vercel settings and restart / redeploy.

19. Glossary

Term	Meaning
Tracker	The recruiter's Excel file with one row per candidate.
CTC / Lpa	Cost to company (annual salary) in lakhs per annum.
LWD / DOJ	Last working day / date of joining.
Text mode / file mode	Whether a resume was sent to Gemini as extracted text or as the PDF itself (scanned).
Facts	The JSON Gemini returns for one resume, before any formatting.
Mapping	Which tracker column holds which field, e.g. `current_ctc` → column E.
Evidence	For each output value: where it came from and whether it was found in the source.
Quota day	The 24 hours between two resets of Gemini's free daily quota (midnight US Pacific time).